

Pregunta 1

Correcta

Se puntúa 3,00000 sobre 3,00000

Dado el siguiente código:

```

int enigma(n)
1.  if  $n \% 4 \neq 0$  or  $n \leq 1$ 
2.      return 1
3.   $k = 4 * enigma(n/4) + 6 * enigma(n/4)$ 
4.  for  $i = 1, i \leq n, i++$ 
5.       $k = k + i$ 
6.       $j = n$ 
7.      while  $j > 1$ 
8.           $k = k + i + j$ 
9.           $j = j/24$ 
10. for  $i = 1, i \leq 4, i++$ 
11.     for  $j = 1, j \leq 4, j++$ 
12.         if  $i * j == 4$ 
13.              $k = i * enigma(n/4) + j * enigma(n/4)$ 
14. return k

```

a) (1/5) El peor caso para el algoritmo ocurre cuando n es: ✓

b) (2/5) La recurrencia más apropiada que describe el tiempo de ejecución en el peor caso es:

- ☐ $10T(n/4) + \Theta(n \lg n)$
- ☒ $8T(n/4) + \Theta(n \lg n)$ ✓
- ☐ $16T(n/4) + \Theta(n \log n)$
- ☐ $18T(n/4) + \Theta(n^2)$
- ☐ $12T(n/4) + \Theta(n^2)$
- ☐ $10T(n/4) + \Theta(n^2)$
- ☐ $18T(n/4) + \Theta(n \log n)$
- ☐ $12T(n/4) + \Theta(n \lg n)$
- ☐ $16T(n/4) + \Theta(n^2)$
- ☐ $4T(n/4) + \Theta(n \log n)$
- ☐ $4T(n/4) + \Theta(n^2)$
- ☐ $8T(n/4) + \Theta(n^2)$

La respuesta correcta es: $8T(n/4) + \Theta(n \lg n)$

c) (2/5) El orden asintótico de esta recurrencia es:

- ☐ $T(n) = \Theta(n)$
- ☐ $T(n) = \Theta(n \log n)$
- ☐ $T(n) = \Theta(n^{\log_4 12})$
- ☐ $T(n) = \Theta(n^2)$
- ☐ $T(n) = \Theta(n^{\log_4 10})$
- ☐ $T(n) = \Theta(n^{\log_4 18})$
- ☒ $T(n) = \Theta(n^{\log_4 8})$ ✓

La respuesta correcta es: $T(n) = \Theta(n^{\log_4 8})$

Pregunta 2

Parcialmente correcta

Se puntúa 2,14286 sobre 3,00000

El siguiente pseudocódigo corresponde a un algoritmo que sigue la estrategia de Dividir y Conquistar para calcular el número de elementos divisibles por 5 que hay en el arreglo $A[p..r]$. En este, el problema se divide en dos subproblemas.

```

int numDivisible5(A,p,r):
1.  if _____[A]
2.    if _____[B]
3.      return 1
4.    else
5.      return _____[C]
6.  else
7.     $q = \lfloor (p+r)/2 \rfloor$ 
8.    return _____[D]

```

A. Complete los espacios [A], [B], [C] y [D] de este pseudocódigo en las siguientes preguntas, siguiendo los siguientes criterios y notación:

- No poner ningún espacio ni ;
- Respetar la diferencia entre mayúsculas y minúsculas
- No incluir nuevas variables
- Al llamar una función, poner el nombre exacto con los argumentos entre paréntesis separados por comas. Por ejemplo, suma(A,i,j)
- Utilizar expresiones (posiblemente compuestas) de acuerdo a la siguiente tabla:

Operación	Notación (sin espacios)
Sumar a y b	$a + b$
Restar b de a	$a - b$
Multipliación de a y b	$a * b$
Dividir a entre b	a/b
a módulo (residuo) b	$a \% b$
Máximo de a y b	$\max(a, b)$
Mínimo de a y b	$\min(a, b)$
Verificar si a y b son iguales	$a == b$
Verificar si a y b son diferentes	$a != b$
Verificar si a es menor o igual a b	$a <= b$
Verificar si a es menor que b	$a < b$
Verificar si a es mayor o igual a b	$a >= b$
Verificar si a es mayor que b	$a > b$
i -ésimo elemento del arreglo A	$A[i]$

Para cumplir con lo especificado, llenar lo correspondiente a cada línea:

[A] (1/7): ✓

[B] (1/7): ✓

[C] (1/7): ✓

[D] (2/7): ✗

B. Responda las siguientes preguntas:

a) (1/7) La ecuación de recurrencia del tiempo de ejecución del algoritmo es:

- ☐ $T(n) = 2T(n/2) + \Theta(n \log n)$
☐ $T(n) = 2T(n/2) + \Theta(n^2)$
☐ $T(n) = 2T(n/2) + \Theta(n)$
☒ $T(n) = 2T(n/2) + \Theta(1)$ ✓

La respuesta correcta es: $T(n) = 2T(n/2) + \Theta(1)$

b) (1/7) El tiempo de ejecución de este algoritmo es:

- ☐ $\Theta(n \log n)$
☐ $\Theta(n^2)$
☐ $\Theta(\log n)$
☐ $\Theta(1)$
☒ $\Theta(n)$ ✓

La respuesta correcta es: $\Theta(n)$

Pregunta **3**

Correcta

Se puntúa 1,00000 sobre 1,00000

Si $f(n) = n^3 + 8n^2 + 3n$ y $g(n) = 2n^3 \lg n - 2n$, entonces:

Seleccione una o más de una:

- ☐ a. $g(n) \in o(f(n))$ y $f(n) \in O(g(n))$
☒ b. $g(n) \notin o(f(n))$ y $f(n) \notin o(f(n))$
☒ c. $f(n) \in o(g(n))$ y $f(n) \notin \Omega(g(n))$
☐ d. $g(n) \in \omega(g(n))$ y $f(n) \in O(g(n))$



Las respuestas correctas son: $f(n) \in o(g(n))$ y $f(n) \notin \Omega(g(n))$
 $g(n) \notin o(f(n))$ y $f(n) \notin o(f(n))$

Pregunta 4

Incorrecta

Se puntúa 0,00000 sobre 1,00000

Escoja las dos respuestas correctas:

- ☒ a. El problema del agente viajero se puede resolver en tiempo lineal con un algoritmo no determinístico. ✓
- ☐ b. El problema de las torres de Hanoi se puede resolver en tiempo lineal con un algoritmo no determinístico.
- ☒ c. Los problemas NP-Completo se encuentran dentro de los problemas intratables. ✗
- ☐ d. Ya se encontraron soluciones óptimas para el problema de las torres de Hanoi.

Las respuestas correctas son:

Ya se encontraron soluciones óptimas para el problema de las torres de Hanoi.,

El problema del agente viajero se puede resolver en tiempo lineal con un algoritmo no determinístico.

Pregunta 5

Correcta

Se puntúa 1,00000 sobre 1,00000

Dado el siguiente código

```

Opera(int B, int N) {
1   int A, C, D;

2   D=0;
3   A=1;
4   C=N;
5   while ( C >= 1 ){
6       A=A*B;
7       D=D+A;
8       C--;
9   }
10  System.out.println(" D=" +D);
11 }

```

Seleccione todas las invariantes de ciclo que se cumplan al inicio de cada iteración:

Seleccione una o más de una:

- ☒ a. Al comienzo de cada iteración, $D = \sum_{k=1}^{N-C} B^k$. ✓
- ☐ b. Al comienzo de cada iteración, $D = \sum_{k=1}^{N-C+1} B^k$.
- ☐ c. Al comienzo de cada iteración, $A = B^{N-C-1}$.
- ☒ d. Al comienzo de cada iteración, $A = B^{N-C}$. ✓
- ☐ e. Al comienzo de cada iteración, $A = B^{N-C+1}$.
- ☐ f. Al comienzo de cada iteración, $D = \sum_{k=1}^{N-C-1} B^k$.

Las respuestas correctas son: Al comienzo de cada iteración, $A = B^{N-C}$.
 , Al comienzo de cada iteración, $D = \sum_{k=1}^{N-C} B^k$.

Pregunta 6

Correcta

Se puntúa 5,00000 sobre 5,00000

Dibuje el árbol de recursión de la recurrencia $T(n) = 7T(n/6) + cn^2$ y, con base en este, responda las siguientes preguntas.

Nota. Recuerde que el nivel 0 del árbol corresponde a la raíz.

Para $n=78364164096$, responda en valor numérico el resultado de las siguientes preguntas:

1. El tamaño de cada subproblema en el nivel 10 del árbol es: ✓
2. El costo de cada subproblema en el nivel 10 del árbol es: ✓ c
3. El número de subproblemas en el nivel 10 del árbol es: ✓
4. Las hojas del árbol están en el nivel: ✓

Con base en el análisis del árbol recursivo se puede concluir que:

1. El número de ejecuciones del caso base es:

- ☒ $O(n^{\log_6 7})$ ✓
☐ $O(n \log n)$
☐ $O(n^2)$
☐ $O(n)$
☐ $O(n^2 \log n)$
☐ $O(n^{\log_2 3})$

La respuesta correcta es: $O(n^{\log_6 7})$

2. La complejidad de la parte recursiva del árbol es:

- ☒ $O(n^2)$ ✓
☐ $O(n)$
☐ $O(n \log n)$
☐ $O(n^2 \log n)$
☐ $O(n^{\log_2 3})$
☐ $O(n^{\log_6 7})$

La respuesta correcta es: $O(n^2)$

3. Basado en el árbol, se procede a probar mediante el método de sustitución la siguiente desigualdad:

- ☒ $T(n) \leq \frac{36}{29}cn^2$ ✓
☐ $T(n) \leq 2cn^{\log_2 3} - 2cn$
☐ $T(n) \leq cn^2 - cn$
☐ $T(n) \leq cn^2 \log n$

La respuesta correcta es: $T(n) \leq \frac{36}{29}cn^2$

Pregunta 7

Correcta

Se puntúa 1,00000 sobre 1,00000

Escoja las dos respuestas correctas:

Seleccione una o más de una:

- ☐ a. Si el tiempo de ejecución de un algoritmo en el caso general es $\Theta(n^2)$, es posible que el tiempo para algunas entradas sea $O(n)$.
- ☐ b. Si el tiempo de ejecución en el peor caso de un algoritmo es $O(n)$, entonces el algoritmo puede finalizar en tiempo $\Omega(n^2)$ sobre algunas entradas de tamaño n .
- ☒ c. Si el tiempo de ejecución en el peor caso de un algoritmo es $O(n^2)$, entonces el algoritmo debe finalizar en tiempo $O(n^2)$ para todas las entradas de tamaño n . ✓
- ☒ d. Si el tiempo de ejecución de un algoritmo en el peor caso es $o(n^2)$, dicho algoritmo puede correr en tiempo $O(n)$ para algunas entradas de tamaño n . ✓

Las respuestas correctas son: Si el tiempo de ejecución de un algoritmo en el peor caso es $o(n^2)$, dicho algoritmo puede correr en tiempo $O(n)$ para algunas entradas de tamaño n .

, Si el tiempo de ejecución en el peor caso de un algoritmo es $O(n^2)$, entonces el algoritmo debe finalizar en tiempo $O(n^2)$ para todas las entradas de tamaño n .

Pregunta 8

Correcta

Se puntúa 1,00000 sobre 1,00000

Dados los siguientes ciclos,

```

i = 0
while i < n/3:
    j = 1
    while j < n:
        sum = sum + 1
        j = 2 * j
    i = i + 1

```

```

i = 0
while i < n/4:
    j = 0
    while j < i:
        sum = sum + 1
        j = j + 1
    i = i + 1

```

Los enunciados que describen acertadamente el comportamiento de cada uno de estos fragmentos de código son:

Seleccione una o más de una:

- ☒ a. $T(n) = \Theta(n \log n)$
- ☐ b. $T(n) = \Theta(n)$
- ☐ c. $T(n) = \Theta(\log n)$
- ☒ d. $T(n) = \Theta(n^2)$



Las respuestas correctas son: $T(n) = \Theta(n^2)$
 $, T(n) = \Theta(n \log n)$

Pregunta 9

Correcta

Se puntúa 1,00000 sobre 1,00000

El número de comparaciones (de elementos del arreglo) que hace InsertionSort para ordenar el arreglo $\langle 44, 58, 48, 76, 58, 75 \rangle$ esRespuesta: 

La respuesta correcta es: 8

Pregunta **10**

Correcta

Se puntúa 1,00000 sobre 1,00000

Un algoritmo resuelve un problema en $f(n) = n^2$ microsegundos (1 seg = 10^6 microsegundos). El tamaño más grande que puede resolver el algoritmo en 1 segundo es: (escriba todas las cifras del número)

Respuesta: 

La respuesta correcta es: 1000

[◀ Programación Dinámica - Factorial](#)

[Práctica 1.1 ▶](#)